# TEMA 8 – PROGRAMACION DE BASE DE DATOS PLSQL. (Cursores y Disparadores)

**INDICE**


1. **CURSORES EXPLÍCITOS.**

   **1.1 ATRIBUTOS DEL CURSOR.**

   **1.2 CURSOR FOR . . LOOP.**

   **1.3 USO DE ALIAS EN CURSORES.**

   **1.4 CURSORES CON PARÁMETROS.**

2. **TRIGGER**

# 1.- CURSORES EXPLÍCITOS.

Hasta el momento hemos venido utilizando cursores implícitos. Este tipo de cursor es muy sencillo y cómodo de usar, pero plantea diversos problemas.

El más importante es que la subconsulta debe devolver una sola fila, de lo contrario, se produciría un error. Por ello, dado que normalmente una consulta devolverá varias filas, se suelen manejar cursores explícitos.

Hay cuatro operaciones básicas para trabajar con un cursor explícito:

1. **Declaración del cursor.** El cursor se declara en la zona de declaraciones según el siguiente formato:

**CURSOR <nombrecursor> IS SELECT <sentencia SELECT>;**

2. **Apertura del cursor.** En la zona de instrucciones hay que abrir el cursor:

**OPEN <nombrecursor>;**

Al hacerlo se ejecuta automáticamente la sentencia SELECT asociada y sus resultados se almacenan en las estructuras internas de memoria manejadas por el cursor. Para acceder a la información debemos dar el paso siguiente.

3. **Recogida de información.** Para recoger la información almacenada en el cursor utilizamos el siguiente formato:

**FETCH <nombrecursor> INTO {<variable_registro> | <listavariables>};**

Después del INTO figurará una variable que recogerá la información de todas las columnas de una tabla (en este caso, la variable puede ser declarada con %ROWTYPE) o una lista de variables, cada una recogerá la columna correspondiente de la orden SELECT. Por tanto serán del mismo tipo que las columnas. Cada FETCH recupera una fila y el cursor avanza automáticamente a la fila siguiente.

4. **Cierre del cursor.** Cuando el cursor no se va a utilizar hay que cerrarlo:

**CLOSE <nombrecursor>;**

El cursor es un identificador, no una variable. Solamente se puede usar para hacer referencia a una consulta. No se le pueden asignar valores ni utilizar en expresiones.

```
DECLARE
    CURSOR cur1 IS
            SELECT dnombre, loc FROM depart;
    v_nombre VARCHAR2 (14);
    v_localidad VARCHAR2 (14);

BEGIN
    OPEN cur1;
    FETCH cur1 INTO v_nombre, v_localidad;
    WHILE cur1%FOUND LOOP
            DBMS_OUTPUT.PUT_LINE (v_nombre || '*' || v_localidad);
            FETCH cur1 INTO v_nombre, v_localidad;
    END LOOP;
    CLOSE cur1;
END;
```

## 1.1 ATRIBUTOS DEL CURSOR.

Para conocer detalles respecto a la situación del cursor hay cuatro atributos para consultar:

- **%FOUND.** Devuelve verdadero si el último FETCH ha recuperado algún valor; en caso contrario, devuelve falso. Si el cursor no estaba abierto devuelve error, y si estaba abierto pero no se había ejecutado aún ningún FETCH, devuelve NULL. Se suele utilizar como condición de continuación en bucles para recuperar información.

- **%NOTFOUND.** Hace lo contrario que el atributo anterior.

- **%ROWCOUNT.** Devuelve el número de filas recuperadas hasta el momento por el cursor (número de FETCH realizados satisfactoriamente).

- **%ISOPEN.** Devuelve verdadero si el cursor está abierto.

## 1.2 CURSOR FOR . . LOOP.

Esta estructura simplifica las tareas realizadas por el cursor. Su formato es el siguiente:

1. **Se declara la información cursor** en la sección correspondiente.

2. **Se procesa el cursor** utilizando el siguiente formato:
   FOR nombrevareg IN nombrecursor LOOP
   …;
   END LOOP;

Al entrar en el bucle, se abre el cursor de manera automática, se declara implícitamente la variable *nombrevareg* de tipo *nombrecursor*%ROWTYPE y se ejecuta el primer FETCH, cuyo resultado quedará en *nombrevareg*.

A continuación se realizarán las acciones que correspondan hasta llegar al END LOOP, que sube de nuevo al FOR … LOOP ejecutando el siguiente FETCH, y depositando otra vez el resultado en *nombrevareg* y así sucesivamente, hasta procesar la última fila de la consulta, momento en el que se producirá la salida del bucle y se cerrará automáticamente el cursor.

La variable *nombrevareg* es local al bucle, por tanto, al salir del bucle no estará disponible.

Dentro del bucle se puede hacer referencia a la variable de registro y a sus campos (cuyo nombre se corresponde con las columnas de la consulta) usando la notación del punto.

```
DECLARE
   CURSOR cur_emple IS
         SELECT apellido, fecha_alt FROM emple
         ORDER BY fecha_alt;
BEGIN
   FOR v_reg_emp IN cur_emple LOOP
      DBMS_OUTPUT.PUT_LINE (v_reg_emp.apellido || '*' ||
      v_reg_emple.fecha_alt);
   END LOOP;
END;
```

## 1.3 USO DE ALIAS EN CURSORES.

Cuando en las consultas de un cursor se usan funciones de grupo, hay que utilizar alias para dar nombre a esas funciones de grupo.

## 1.4 CURSORES CON PARÁMETROS.

Un cursor puede tener parámetros. En este caso se aplicará el siguiente formato genérico:

**CURSOR *nombrecursor [ (parámetro1, parámetro2,…)]***
**IS SELECT <sentencia select en la que intervendrán los parámetros>;**

Los parámetros tienen la siguiente sintaxis:

***Nombredevariable* [IN] *tipodedato* [{ := | DEFAULT } *valor* ]**

El ámbito de estos parámetros es local al cursor.

```
DECLARE
    …
    CURSOR cur1 ( v_departamento NUMBER,
                  v_oficio VARCHAR2 DEFAULT 'DIRECTOR' )
          IS SELECT apellido, salario FROM emple
          WHERE dept_no = v_departamento AND oficio = v_oficio;
```

Para abrir un cursor pasándole parámetros lo haremos así:

OPEN *nombrecursor* [ ( ***parámetro1, parámetro2, …*** ) ];

Donde parámetro1, parámetro2, son expresiones que contienen los valores que se pasarán al cursor. No tienen por qué ser los mismos nombres de las variables indicadas como parámetros al declarar el cursor.

```
DECLARE

    CURSOR cur1 ( v_departamento NUMBER,
                  v_oficio VARCHAR2 DEFAULT 'DIRECTOR' )
          IS SELECT apellido, salario FROM emple
          WHERE dept_no = v_departamento AND oficio = v_oficio;
…
Cualquiera de los siguientes comandos abrirá el cursor:
BEGIN
…
OPEN cur1 (v_departamento);
OPEN cur1 (v_departamento, v_oficio);
OPEN cur1 (20, 'VENDEDOR');
```

**Ejemplo: Mostrar por pantalla todos los departamentos de la localidad de Madrid**

```
Nombre     INVESTIGACION
Localidad  MADRID
```

Forma 1:

```
DECLARE
    CURSOR cur1 IS
          SELECT dnombre, loc FROM depart WHERE loc='MADRID';
```

```
        v_nombre VARCHAR2 (14);
        v_localidad VARCHAR2 (14);

    BEGIN
        OPEN cur1;
        FETCH cur1 INTO v_nombre, v_localidad;
        WHILE cur1%FOUND LOOP
                DBMS_OUTPUT.PUT_LINE (' Nombre    ');
                DBMS_OUTPUT.PUT_LINE (v_nombre);
                DBMS_OUTPUT.PUT_LINE (' ');
                DBMS_OUTPUT.PUT_LINE (' Localidad ');
                DBMS_OUTPUT.PUT_LINE (v_localidad);
        FETCH cur1 INTO v_nombre, v_localidad;
        END LOOP;
        CLOSE cur1;
    END;
```

Forma 2:

```
    DECLARE
        CURSOR cur1 IS
                SELECT dnombre, loc FROM depart;
        v_nombre VARCHAR2 (14);
        v_localidad VARCHAR2 (14);

    BEGIN
        OPEN cur1;
        FETCH cur1 INTO v_nombre, v_localidad;
        WHILE cur1%FOUND LOOP
                IF v_localidad='MADRID' then
                        DBMS_OUTPUT.PUT_LINE (' Nombre    ');
                        DBMS_OUTPUT.PUT_LINE (v_nombre);
                        DBMS_OUTPUT.PUT_LINE (' ');
                        DBMS_OUTPUT.PUT_LINE (' Localidad ');
                        DBMS_OUTPUT.PUT_LINE (v_localidad);
        END IF;
                FETCH cur1 INTO v_nombre, v_localidad;
        END LOOP;
        CLOSE cur1;
    END;
```

# 2.- TRIGGER.

## DEFINICION

Un Trigger es un programa almacenado, creado para ejecutarse automaticamente cuando ocurra un evento en nuestra base de datos. Dichos eventos son generados por los comandos INSERT, UPDATE y DELETE, los cuales hacen parte del DML.

**El funcionamiento de un disparador o trigger consiste en realizar una operación en una Base de Datos (inserción, borrado o actualización) y reflejar esa operación de algún modo en una tabla (log).**

## CREACION

**CREATE OR REPLACE TRIGGER nombre_trigger**
BEFORE/AFTER (antes o después)
UPDATE OR DELETE OR INSERT
ON tabla
OF Campo de la tabla
FOR EACH ROW (para cada fila)
**BEGIN**
    **/* cuerpo del trigger */**
**END;**

## CONTROL DE EXCEPCIONES EN LOS TRIGGER CON RAISE

**SINTAXIS**

RAISE_APPLICATION_ERROR(numeroerror, mensaje de error)

**EJEMPLO**

RAISE_APPLICATION_ERROR(-20001,' el empleado no existe')

el número de error tiene que estar comprendido entre -20001 y -20999

## IDENTIFICADORES NEW Y OLD EN TRIGGERS

- Si queremos relacionar el trigger con columnas especificas de una tabla debemos usar los identificadores OLD y NEW.
- Con ellos podemos referenciar a los valores antes y después de la actualización a nivel de fila
- Solo se puede utilizar a nivel de fila
- Se utiliza :new.columna :old.columna
- Ejemplo: if :new.columna<:old.columna
- Cuando hacemos DELETE :new.columna es NULL
- Cuando hacemos INSERT :old.columna es NULL
- Cuando hacemos UPDATE ambas tienen valor
- Cuando utilicemos new y old la clausuala WHEN del trigger no hace falta
- OLD indica el valor antiguo de la columna y NEW el valor nuevo que pudiese tomar. Por ejemplo: OLD.idproducto ó NEW.idproducto.
- Si usamos la sentencia UPDATE podremos referirnos a un valor OLD y

NEW, ya que modificaremos registros existentes por nuevos valores.

- En cambio si usamos INSERT solo usaremos NEW, ya que su naturaleza es únicamente de insertar nuevos valores a las columnas.

- Y si usamos DELETE usaremos OLD debido a que borraremos valores que existen con anterioridad.

## BEFORE Y AFTER

Estas clausulas indican si el Trigger se ejecuta **antes o después** del evento DML. Hay ciertos eventos que no son compatibles con estas sentencias.

Por ejemplo,

- si tuvieras un Trigger AFTER que se ejecuta en una sentencia UPDATE, sería ilógico editar valores nuevos NEW, sabiendo que el evento ya ocurrió.

- Igual sucedería con la sentencia INSERT, el Trigger tampoco podría referenciar valores NEW, ya que los valores que en algún momento fueron NEW, han pasado a ser OLD.

## ¿QUÉ UTILIDADES TIENEN LOS TRIGGERS?

- Con los Triggers podemos implementar varios casos de uso que mantengan la integridad de la base de datos, como Validar información, Calcular atributos derivados, Seguimientos de movimientos en la base de datos, etc.

- Cuando surja una necesidad en donde veas que necesitas que se ejecute una acción implícitamente(sin que la ejecutes manualmente) sobre los registros de una tabla, entonces puedes considerar el uso de un Trigger.

## CONSULTAS SENCILLAS A TRIGGER PARA VER LOS QUE TENEMOS

```
select trigger_name, triggering_event, status
  from user_triggers
  where trigger_name like 'TR%XXX';

select trigger_name, triggering_event, status
  from user_triggers
  where table_name = 'XXX';
```

## VER LA INFORMACIÓN DE UN TRIGGER EN MYSQL

Se usa el comando SHOW CREATE TRIGGER, como por ejemplo:

**SHOW CREATE TRIGGER futbolista_ai_trigger;**

También puedes ver los Triggers que hay en tu base de datos con:

**SHOW TRIGGERS;**

## ELIMINAR UN TRIGGER EN MYSQL

DROP, DROP y más DROP. Como ya sabes usamos este comando para eliminar casi cualquier cosa en nuestra base de datos:

**DROP TRIGGER [IF EXISTS] nombre_trigger**

Recuerda que podemos adicionar la condicion IF EXISTS para indica que si el Trigger ya existe, entonces que lo borre.

## EJEMPLO DE TRIGGER BEFORE EN LA SENTENCIA UPDATE

Queremos hacer un Trigger que valide la edad de un cliente antes de una sentencia UPDATE. Si por casualidad el nuevo valor es negativo, entonces asignaremos NULL a este atributo.

```
CREATE TRIGGER cliente_BU_Trigger
BEFORE UPDATE ON cliente FOR EACH ROW
BEGIN
-- La edad es negativa?
IF NEW.edad
```

Este Trigger se ejecuta antes de haber insertado el registro, lo que nos da el poder de verificar primero si el nuevo valor de la edad esta en el rango apropiado, si no es así entonces asignaremos NULL a ese campo.

## EJEMPLO DE TRIGGER AFTER EN LA SENTENCIA UPDATE

Supongamos que tenemos una Tienda de accesorios para Gamers. Para la actividad de nuestro negocio hemos creado un sistema de facturación muy sencillo, que registra las ventas realizadas dentro de una factura que contiene el detalle de las compras. Nuestra tienda tiene 4 vendedores de turno, los cuales se encargan de registrar las compras de los clientes en el horario de funcionamiento.

Queremos hacer un Trigger que guarde los cambios realizados sobre la tabla DETALLE_FACTURA de la base de datos realizados por los vendedores.

```
CREATE TRIGGER detalle_factura_AU_Trigger
AFTER UPDATE ON detalle_factura FOR EACH ROW
BEGIN
	INSERT INTO log_updates (idusuario, descripcion)
	VALUES (user( ), CONCAT('Se modificó el registro ','(',
	OLD.iddetalle,',', OLD.idfactura,',',OLD.idproducto,',',
	OLD.precio,',', OLD.unidades,') por ',
	'(', NEW.iddetalle,',', NEW.idfactura,',',NEW.idproducto,',',
	NEW.precio,',', NEW.unidades,')'));
END
```

Con este registro de logs podremos saber si algún vendedor “ocioso” esta alterando las facturas, lo que lógicamente sería atentar contra las finanzas de nuestro negocio. Cada registro nos informa el usuario que modificó la tabla DETALLE_FACTURA y muestra una descripción sobre los cambios en cada columna.

## EJEMPLO DE TRIGGER BEFORE EN LA SENTENCIA INSERT

El siguiente ejemplo muestra como mantener la integridad de una base de datos con respecto a un atributo derivado.

Supón que tienes una Tienda de electrodomésticos y que has implementado un sistema de facturación. En la base de datos que soporta la información de tu negocio, existen varias tablas, pero nos vamos a centrar en la tabla PEDIDO y la tabla TOTAL_VENTAS.

TOTAL_VENTAS almacena las ventas totales que se le han hecho a cada cliente del negocio. Es decir, si el cliente Armado Barreras en una ocasión compró 1000 dolares, luego compró 1250 dolares y hace poco ha vuelto a comprar 2000 dolares, entonces el total vendido a este cliente es de 4250 dolares.

Pero supongamos que eliminamos el ultimo pedido hecho por este cliente, ¿que pasaría con el registro en TOTAL_VENTAS uedaría desactualizado.

Usaremos tres Triggers para solucionar esta situación. Para que cada vez que usemos un comando DML en la tabla PEDIDO, no tengamos que preocuparnos por actualizar manualmente TOTAL_VENTAS.

**-- TRIGGER PARA INSERT**

```
CREATE TRIGGER PEDIDO_BI_TRIGGER
BEFORE INSERT ON PEDIDO
FOR EACH ROW
```

```
BEGIN
        DECLARE cantidad_filas INT;
        SELECT COUNT(*)
        INTO cantidad_filas
        FROM TOTAL_VENTAS
        WHERE idcliente=NEW.idcliente;

        IF cantidad_filas > 0 THEN
                UPDATE TOTAL_VENTAS
                SET total=total+NEW.total
                WHERE idcliente=NEW.idcliente;
        ELSE
                INSERT INTO TOTAL_VENTAS (idcliente,total)
                                VALUES(NEW.idcliente,NEW.total);
        END IF;
END//
```

**-- TRIGGER PARA UPDATE**

```
CREATE TRIGGER PEDIDO_BU_TRIGGER
BEFORE UPDATE ON PEDIDO
FOR EACH ROW
BEGIN
        UPDATE TOTAL_VENTAS
        SET total=total+(NEW.total-OLD.total)
        WHERE idcliente=NEW.idcliente;
END//
```

**-- TRIGGER PARA DELETE**

```
CREATE TRIGGER PEDIDO_BD_TRIGGER
BEFORE DELETE ON PEDIDO
FOR EACH ROW
BEGIN
        UPDATE TOTAL_VENTAS
        SET total=total-OLD.total
        WHERE idcliente=OLD.idcliente;
END//
```

Con todos ellos mantendremos el total de ventas de cada cliente actualizado dependiendo del evento realizado sobre un pedido.

Si insertamos un nuevo pedido generado por un cliente existente, entonces vamos rápidamente a la tabla TOTAL_VENTAS y actualizamos el total comprado por ese cliente con una sencilla suma acumulativa.

Ahora, si cambiamos el monto de un pedido, entonces vamos a TOTAL_VENTAS para descontar el monto anterior y adicionar el nuevo monto.

Y si eliminamos un pedido de un cliente simplemente descontamos del total acumulado el monto que con anterioridad habíamos acumulado.